NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

SC1007/CE1007/CZ1007

Stacks



Presented by Dr Owen Noel Newton Fernando

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Today, we're going to explore **stacks**, one of the fundamental linear data structures in computer science. We'll look at how stacks organise data, how their core operations work, and how they can be implemented using linked structures in Python.



Objectives

- Introduce the Stack data structure and the Last In, First Out (LIFO) principle.
- Explain the role of the top and the organisation of elements in a stack.
- Examine the core operations: push, pop, peek, is_empty, and get_size.
- Explore the implementation of a stack directly using linked nodes.
- Examine how a stack can be implemented using a LinkedList class.
- Explore common applications of stacks in computer science.



© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Our focus is to develop a clear understanding of stacks, beginning with the Last In, First Out principle and the role of the top. We'll see how this organisation determines the way elements can be accessed and processed.

We'll examine the main stack operations, including push, pop, peek, is_empty, and get_size. Rather than simply learning the methods, we'll look at what happens to the underlying structure when each operation is performed.

We'll then implement a stack directly using linked nodes and also by reusing a LinkedList class. Towards the end, we'll explore several computing applications where this LIFO behaviour is naturally useful.

Learning Outcome

- Explain the LIFO principle and how a stack organises data.
- Describe and trace push, pop, peek, is_empty, and get_size operations.
- Implement a Stack directly using linked nodes.
- Implement a Stack using an existing LinkedList class.
- Analyse how stack operations modify the top element and stack size.
- Identify and explain common applications of stacks in computing.

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

By the end of the session, you should be able to explain how a stack behaves, rather than simply remember that it follows LIFO. You should also be able to trace the core operations and determine how each one affects the top element and the size of the stack.

From a programming perspective, you'll build stacks in two different ways: first directly using linked nodes, and then by making use of an existing LinkedList class. This will help you see how the same Stack ADT can be supported by different underlying implementations.

You should also be able to recognise computing problems where stack behaviour is appropriate. This includes situations involving function calls, recursion, expression processing, and backtracking.

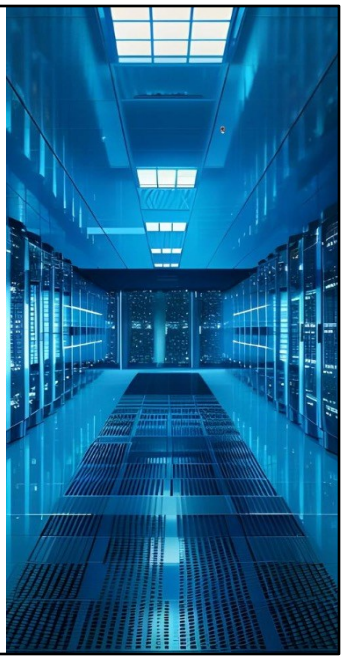
Contents



- Introduction to Stacks
- Implementing a Stack Using Linked Nodes
- Working Example Using Linked Nodes
- Implementing a Stack Using a LinkedList Class
- Working Example Using a LinkedList Class
- Stack Applications
- Conclusion

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted



We'll begin by introducing stacks and looking at how the LIFO principle determines the way elements are organised and accessed. We'll then move from this basic idea to the main operations that define stack behaviour.

Next, we'll look at two ways of implementing a stack. We'll first build one directly using linked nodes, and then implement the same Stack ADT using an existing LinkedList class, with working examples to see how each approach operates.

Finally, we'll explore some common applications of stacks in computing and bring the main ideas together. As we move through the lecture, notice how the behaviour of the Stack remains the same even when its underlying implementation changes.

Introduction to Stacks

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Stacks provide a simple way of organising data when the order of access matters. Before looking at how we implement them in Python, we'll first understand how a stack stores and accesses elements, and why it follows the Last In, First Out, or LIFO, principle.

What is a Stack?



- A **Stack** is a linear data structure that follows the **Last In, First Out (LIFO)** principle.
- The **last element added** is the **first element removed**.
- Elements are added and removed from one end, called the **top**.
- A new element is placed on the **top** of the stack.
- Removing an element takes the **current top element**.
- Therefore, elements are removed in the **reverse order of insertion**.



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

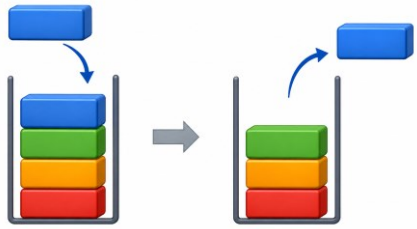
A stack is a linear data structure where the order in which elements are removed depends on the order in which they were added. It follows Last In, First Out, or LIFO, meaning the most recently added element is always the first one to come out.

Unlike structures where we can access elements at different positions, a stack works from just one end, called the top. Whenever we add a new element, it becomes the new top, and removal also happens from this same position.

As a result, elements come out in the reverse order of insertion. This simple access rule defines how a stack behaves, regardless of how we later choose to implement it.

Understanding LIFO Behaviour

- **LIFO** stands for **Last In, First Out**.
- The most recently added element always becomes the **top element**.
- Only the **top element** is directly available for removal.
- Removing the top element exposes the element immediately below it.
- Each addition or removal changes which element is at the **top**.
- This access pattern distinguishes stacks from other linear data structures.



© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Think of LIFO as controlling which element we're allowed to access next. Whenever a new element is added, it goes above the existing elements and immediately becomes the new top of the stack.

The diagram shows what happens when that top element is removed. We can't directly take an element from the middle; we remove the top first, which then exposes the element immediately underneath it.

So the top changes whenever we add or remove an element, while the LIFO rule remains unchanged. This restricted access is what gives a stack its distinctive behaviour compared with other linear data structures.

Why Do We Use Stacks?

- Stacks are useful when data needs to be processed in reverse order.
- They naturally support matching nested structures, such as parentheses and HTML tags.
- They help manage the execution of function and recursive calls.
- Each active function call can be maintained until it completes and returns.
- Stacks are widely used in arithmetic expression processing and evaluation.
- Their LIFO behaviour provides a simple way to manage data when the most recently stored item must be processed first.



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Think about what happens when the most recent piece of information needs to be handled before anything that came earlier. A stack gives us exactly this behaviour through its Last In, First Out ordering, which makes it suitable for several common programming tasks.

For example, stacks can reverse data naturally and keep track of nested structures such as parentheses or HTML tags. They also play a central role in function calls, where information about each active call is retained until that function finishes and control returns to the previous call.

This becomes particularly useful with recursion, because several calls may be active at the same time. Stacks are also used when processing arithmetic expressions, where operands and intermediate results need to be managed in a well-defined order.

Implementing a Stack Using Linked Nodes

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

We'll now turn from how a stack behaves to how we can actually build one in Python. Our first implementation will use linked nodes, where each node stores an element and a reference to the next node in the stack.

Implementing a Stack Using Linked Nodes

```
class Node:
    def __init__(self, data):
        self.data = data     # stores the data
        self.next = None     # points to next node
```

```
class Stack:
    def __init__(self):
        self.top = None      # points to top node
        self.size = 0        # tracks number of nodes
```

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted



Here, we begin with two classes that provide the basic structure for our implementation. The Node class stores the actual data and a next reference, which will later allow each node to connect to the node below it in the stack.

The Stack class then keeps track of the structure through two attributes. top holds a reference to the current top node, while size records how many nodes are currently stored.

When a stack is first created, there are no elements, so top is None and size is zero. These two attributes will be updated as we perform stack operations such as push and pop.

Core Stack Operations

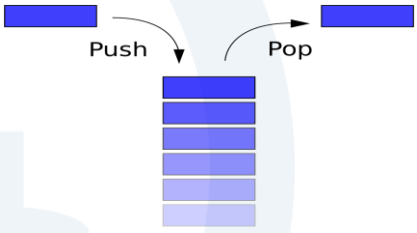
`peek()`: Returns the top element without removing it

`push()`: Adds an element to the top of the stack

`pop()`: Removes and returns the top element

`is_empty()`: Checks if the stack is empty

`get_size()`: Returns the current size of the stack



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

To work with a stack, we only need a small set of operations, and each one has a clearly defined role. Some operations change the contents of the stack, while others simply give us information about its current state.

The two operations you'll use most often are push() and pop(). push() places a new element at the top, while pop() removes that top element and returns it; peek() lets us inspect the same element without removing it.

We also need to know whether there's anything available to process and how many elements are stored. That's where is_empty() and get_size() come in, and we'll implement each of these operations in the following slides.

Core stack operations: peek()

Returns the top element without removing it

```
def peek(self):
    if self.is_empty():
        raise IndexError("Peek from empty stack")
    return self.top.data
```



- First checks if stack is empty using `is_empty()`
- If empty, raises an `IndexError` with a descriptive message
- If not empty, returns the data from the top node without removing it
- `IndexError`: https://docs.python.org/3/library/exceptions.html

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Suppose we want to see the element at the top of the stack without changing anything. The peek() operation does exactly that, so in this example it would return 40 while leaving all three nodes in their current positions.

Before accessing the top node, the method checks is_empty(), because an empty stack has no value to return. If there's no element available, we raise an IndexError rather than trying to access data that doesn't exist.

For a non-empty stack, self.top.data gives us the required value directly. Notice that neither self.top nor self.size changes, because peek() only observes the stack rather than modifying it.

Core stack operations: push()

```
def push(self, data):
    new_node = Node(data)
    new_node.next = self.top
    self.top = new_node
    self.size += 1
```



- Step 1: Create new node with the data
- Step 2: Link new node to current top
- Step 3: Make new node the top
- Step 4: Increase size count

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

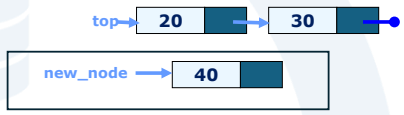
The push() operation adds a new element to the top of the stack, and here we'll follow the process step by step. Suppose the stack currently contains 20 at the top, followed by 30, and we want to insert another value.

The method first creates a new node to hold that value. It then links this new node to the current top before changing self.top, which ensures that we don't lose the connection to the existing nodes.

Once that link is established, the new node becomes the top of the stack. Finally, we increase size by one so that it continues to represent the number of elements stored.

Core stack operations: push()

```
def push(self, data):
    new_node = Node(data)
    new_node.next = self.top
    self.top = new_node
    self.size += 1
```



- **Step 1: Create new node with the data**
- Step 2: Link new node to current top
- Step 3: Make new node the top
- Step 4: Increase size count

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

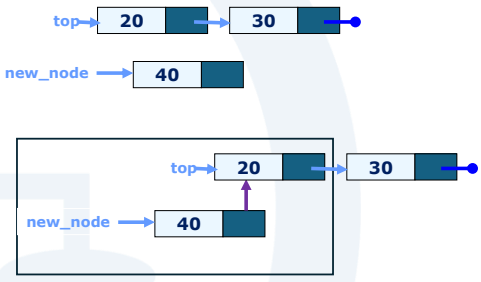
We begin with new_node = Node(data), which creates a new node containing the data we want to add to the stack. In this example, the value is 40, so a new node storing 40 is created.

The variable new_node now refers to this newly created node. Its next reference is initially None, because we haven't connected it to the existing stack yet.

At this stage, the original stack remains exactly as it was, with 20 still at the top followed by 30. The new node containing 40 exists separately, ready to be linked in the next step.

Core stack operations: push()

```
def push(self, data):
    new_node = Node(data)
    new_node.next = self.top
    self.top = new_node
    self.size += 1
```



- Step 1: Create new node with the data
- **Step 2: Link new node to current top**
- Step 3: Make new node the top
- Step 4: Increase size count

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

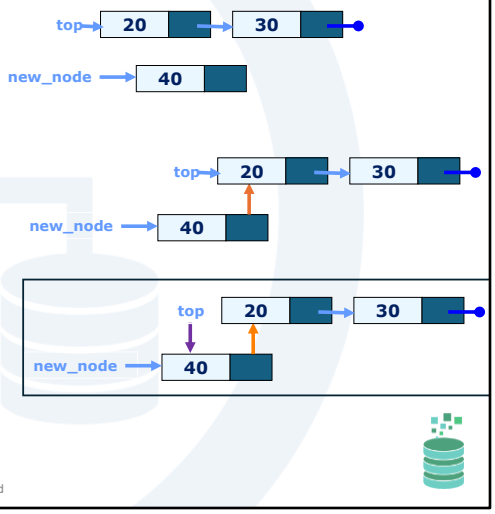
Next, we execute new_node.next = self.top, which connects the new node to the existing stack. The next reference of the node containing 40 is set to the current top node, which contains 20.

At this point, 40 is linked to 20, and 20 remains linked to 30. So we've preserved the complete chain of existing nodes while adding a connection from our new node.

Notice that self.top still points to 20; we haven't changed the top of the stack yet. That happens in the next step, once this connection has been safely established.

Core stack operations: push()

```
def push(self, data):
    new_node = Node(data)
    new_node.next = self.top
    self.top = new_node
    self.size += 1
```



- Step 1: Create new node with the data
- Step 2: Link new node to current top
- **Step 3: Make new node the top**
- **Step 4: Increase size count**

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

With the new node already connected to the existing stack, we execute self.top = new_node. This moves the top reference from the node containing 20 to the new node containing 40, so 40 officially becomes the new top of the stack.

Notice that the connection we created earlier remains unchanged: 40 points to 20, and 20 continues to point to 30. We've therefore inserted the new element without disturbing the rest of the linked structure.

Finally, self.size += 1 increases the number of elements in the stack by one. The push() operation is now complete, with 40 at the top.

Core stack operations: pop()

```
def pop(self):
    if self.is_empty():
        raise IndexError("Pop from empty stack")
    popped_node = self.top
    self.top = self.top.next
    self.size -= 1
    return popped_node.data
```



- **Step 1: Check if stack is empty**
- Step 2: Save current top node
- Step 3: Move top to next node
- Step 4: Decrease size count
- Step 5: Return the data from popped node

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

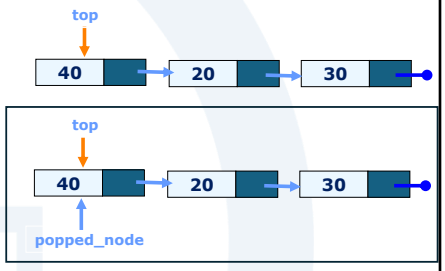
Let's focus on the pop() operation and go through it step by step to see how a node is removed from the top of the stack. Before removing anything, pop() first checks whether the stack is empty using self.is_empty(). This check is necessary because there must be a top element available before we can remove one.

In the example shown, the stack is not empty because top currently refers to the node containing 40. So the condition is false, and the method can continue with the removal process.

If the stack were empty, the method would instead raise an IndexError with the message "Pop from empty stack". This prevents us from attempting to remove an element that doesn't exist.

Core stack operations: pop()

```
def pop(self):
    if self.is_empty():
        raise IndexError("Pop from empty stack")
    popped_node = self.top
    self.top = self.top.next
    self.size -= 1
    return popped_node.data
```



- Step 1: Check if stack is empty
- **Step 2: Save current top node**
- Step 3: Move top to next node
- Step 4: Decrease size count
- Step 5: Return the data from popped node

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

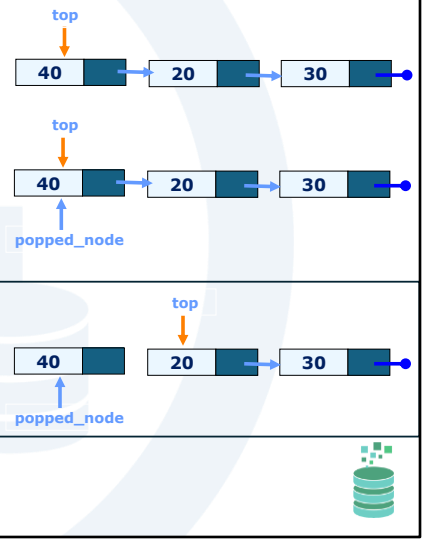
Once we know the stack is not empty, the next step is to save the current top node before we change the stack. The statement popped_node = self.top makes popped_node refer to the same node currently referenced by top.

In this example, both top and popped_node are therefore pointing to the node containing 40. We need to keep this reference because this is the node being removed from the stack.

At this stage, nothing has actually been removed yet; we've simply saved the node that we intend to pop. We'll use this reference later to return its data after the stack has been updated.

Core stack operations: pop()

```
def pop(self):
    if self.is_empty():
        raise IndexError("Pop from empty stack")
    popped_node = self.top
    self.top = self.top.next
    self.size -= 1
    return popped_node.data
```



- Step 1: Check if stack is empty
- Step 2: Save current top node
- **Step 3: Move top to next node**
- Step 4: Decrease size count
- Step 5: Return the data from popped node

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

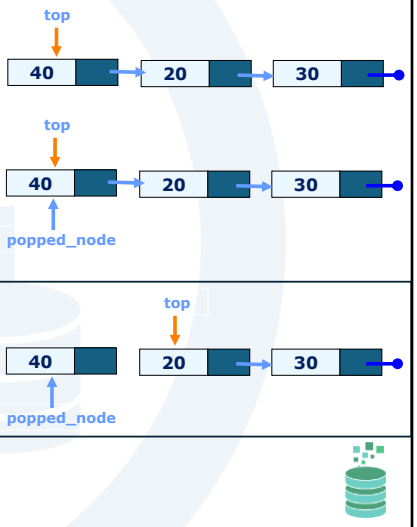
Next, we move top to the next node using self.top = self.top.next. Since the current top node containing 40 points to the node containing 20, top is updated to refer to 20.

Notice that the node containing 40 is no longer part of the stack, but we haven't lost access to it. The variable popped_node still refers to that node because we saved this reference in the previous step.

The stack therefore now begins with 20, followed by 30, while popped_node keeps the removed node containing 40 available so that its data can be returned later.

Core stack operations: pop()

```
def pop(self):
    if self.is_empty():
        raise IndexError("Pop from empty stack")
    popped_node = self.top
    self.top = self.top.next
    self.size -= 1
    return popped_node.data
```



- Step 1: Check if stack is empty
- Step 2: Save current top node
- Step 3: Move top to next node
- **Step 4: Decrease size count**
- **Step 5: Return the data from popped node**

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

With top now pointing to 20, the structural change to the stack is complete. We then execute self.size -= 1 to decrease the size count by one, because the node containing 40 is no longer part of the stack.

The final step is return popped_node.data. Remember that popped_node still refers to the removed node containing 40, so this statement returns the value 40 to whoever called the pop() method.

After these steps, the stack contains 20 followed by 30, with 20 as the new top. So pop() removes the previous top element, updates the stack size, and returns the data stored in the removed node.

After the method returns, popped_node goes out of scope. If no other reference points to the removed node containing 40, Python can automatically reclaim the memory used by that node.

Core stack operations: is_empty()

```
def is_empty(self):
    return self.top is None
```



- This function just checks if the stack is empty by seeing if top points to None.
  - Stack with multiple items:
    - `top -> [40] -> [20] ->[30] ->None`
    - `is_empty() returns False`
  - Empty Stack:
    - `top -> None`
    - `is_empty() returns True`

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The is_empty() operation is very simple because we only need to examine the top reference. The statement return self.top is None directly returns either True or False, so there's no need to traverse the nodes or count the elements.

In the first example, top points to the node containing 40, which is followed by 20 and 30. Since top refers to an actual node rather than None, self.top is None evaluates to False, telling us that the stack contains elements.

For an empty stack, there are no nodes at all, so top points to None. In that case, self.top is None evaluates to True, and is_empty() returns True to indicate that the stack is empty.

Core stack operations: get_size()

```
def get_size(self):
    return self.size
```



- This function returns the current value of **`self.size`**, which tracks the number of items in the stack.
  - Stack with multiple items:
    - top -> [40] -> [20] ->[30] ->None
      size = 3
    - **`get_size() returns 3`**
  - Empty Stack:
    - top -> None
    - size = 0
    - **`get_size() returns 0`**

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The get_size() operation is even simpler because the stack already keeps track of how many elements it contains. Instead of traversing the linked nodes and counting them, the method simply returns the current value stored in self.size.

In this example, the stack contains three nodes: 40, 20, and 30. Since self.size is 3, calling get_size() immediately returns 3, regardless of which node is currently at the top.

Remember that we update self.size whenever the stack changes. A push() increases it by one, while a pop() decreases it by one, so the stored value always represents the current number of elements.

For an empty stack, top is None and self.size is 0. Therefore, calling get_size() simply returns 0, indicating that there are no elements in the stack.

Working Example Using Linked Nodes

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Let’s bring all the stack operations together and see how they work in a single example.

Working Example - 01

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Stack:
    def __init__(self):
        self.top = None
        self.size = 0

    def peek(self):
        if self.is_empty():
            raise IndexError("Peek: Empty stack")
        return self.top.data

    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node
        self.size += 1
```

```
    def pop(self):
        if self.is_empty():
            raise IndexError("Pop: Empty stack")
        popped_node = self.top
        self.top = self.top.next
        self.size -= 1
        return popped_node.data

    def is_empty(self):
        return self.top is None

    def get_size(self):
        return self.size
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

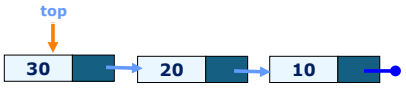
In this working example, we bring together the complete implementation of our linked-node stack. The Node class defines how each individual node stores its data and a reference to the next node.

The Stack class then defines the operations we've just discussed: peek(), push(), pop(), is_empty(), and get_size(). Notice that top identifies the current top node, while size keeps track of how many nodes are currently in the stack.

At this stage, we're only looking at how the complete stack class is organised. On the next slide, we'll create a stack and use these methods step by step, beginning by pushing elements onto the stack.

Working Example – 01 (cont.)

```
34 if __name__ == "__main__":
35
35     s = Stack() # Create a new stack
37
38     s.push(10) # Push some elements:
39     s.push(20)
40     s.push(30)
41
42     print("Size:", s.get_size())    # Print the size. Should print 3
43
44     print("Top element:", s.peek())  # Print top element. Should print 30
45
46     print("Popped:", s.pop())        # Should print 30
47     print("Popped:", s.pop())        # Should print 20
48
49     print("New size:", s.get_size()) # Print new size. Should print 1
50
51     print("New top element:", s.peek()) # Print new top element. Should print 10
52
53     print("Is stack empty?", s.is_empty()) # Check if empty. Should print False
54
55     print("Popped:", s.pop())        # Pop last element. Should print 10
56
57     print("Is stack empty now?", s.is_empty()) # Check if empty again. Should print True
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Here, we're using the stack implementation from the previous slide in a complete working example. The if __name__ == "__main__": block means this part runs when the Python file is executed directly, and we begin by creating an empty stack with s = Stack().

We then push 10, 20, and 30 in that order, so because of LIFO, 30 becomes the top, followed by 20 and 10. Calling get_size() returns 3, while peek() returns 30 without removing anything.

Next, two pop() operations remove 30 and then 20, leaving only 10 in the stack. We can then check the new size and top, test whether the stack is empty, pop the final element, and confirm that is_empty() now returns True.

Working Example – 01 (cont.)

- **Method Call Translation**
  - Create a Stack instance: s = Stack()
  - Call the method: s.push(10)
  - The call can be understood as: Stack.push(s, 10)
  - s is passed to the self parameter.
  - 10 is passed to the data parameter.
- **Inside push()**
  - self refers to Stack instance s.
  - data refers to the value 10.
  - self.top refers to the top-node reference of Stack s.
  - self.size refers to the number of nodes in Stack s.

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

```
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5
6 class Stack:
7     def __init__(self):
8         self.top = None
9         self.size = 0
10
11    def push(self, data):
12        new_node = Node(data)
13        new_node.next = self.top
14        self.top = new_node
15        self.size += 1
16
17 if __name__ == "__main__":
18     # Create a stack instance 's'
19        s = Stack()
20
21        s.push(10)
22        s.push(20)
23        s.push(30)
```

When we write s.push(10), we are calling the push() method on the Stack object referenced by s. One useful way to understand this call is to think of it as Stack.push(s, 10), where Python passes the object s to the self parameter and the value 10 to the data parameter.

So, while this method is executing, self refers to this particular Stack instance, s. That means expressions such as self.top and self.size access the top and size belonging to Stack s.

This connection between s and self will help us follow what happens inside the push() method.

Working Example – 01 (cont.)

- **Operations in push()**
  - For s.push(10):
  - Create a new node:
    new_node = Node(data) creates Node(10).
  - Link to the current top:
    new_node.next = self.top
  - Update the top:
    self.top = new_node makes the new node the top.
  - Increment the size:
    self.size += 1 increases the stack size by 1.
- **Result**: 10 becomes the top element of Stack s.

```
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5
6 class Stack:
7     def __init__(self):
8         self.top = None
9         self.size = 0
10
11    def push(self, data):
12        new_node = Node(data)
13        new_node.next = self.top
14        self.top = new_node
15        self.size += 1
16
17 if __name__ == "__main__":
18     # Create a stack instance 's'
19        s = Stack()
20
21        s.push(10)
22        s.push(20)
23        s.push(30)
```

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

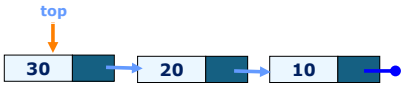
Once s and self are connected, we can focus on what the push() method actually does. For s.push(10), the value of data is 10, so the first statement creates a new node containing that value.

The new node is then linked to whatever node is currently at the top of the stack. After that, self.top is changed to reference the new node, making 10 the new top element.

Finally, the size is increased by one to reflect the additional node. These same four steps are repeated when we later call s.push(20) and s.push(30).

Working Example – 01 (cont.)

```
34 if __name__ == "__main__":
35
35     s = Stack() # Create a new stack
37
38     s.push(10) # Push some elements:
39     s.push(20)
40     s.push(30)
41
42     print("Size:", s.get_size())    # Print the size. Should print 3
43
44     print("Top element:", s.peek())  # Print top element. Should print 30
45
46     print("Popped:", s.pop())        # Should print 30
47     print("Popped:", s.pop())        # Should print 20
48
49     print("New size:", s.get_size()) # Print new size. Should print 1
50
51     print("New top element:", s.peek()) # Print new top element. Should print 10
52
53     print("Is stack empty?", s.is_empty()) # Check if empty. Should print False
54
55     print("Popped:", s.pop())        # Pop last element. Should print 10
56
57     print("Is stack empty now?", s.is_empty()) # Check if empty again. Should print True
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

From line 42 onwards, we start using the remaining stack operations on the stack we have created. First, get_size() returns 3, and peek() returns 30, confirming that there are three elements and that 30 is currently at the top.

Next, we call pop() twice. Because a stack follows LIFO, the first call removes and returns 30, and the second removes and returns 20, leaving only 10; therefore, get_size() returns 1, peek() returns 10, and is_empty() returns False.

Finally, we call pop() once more to remove and return the remaining value 10. At this point there are no nodes left in the stack, top becomes None, and the final call to is_empty() returns True.

Implementing a Stack Using a LinkedList Class

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

So far, we have implemented a stack directly using linked nodes and explored each of its core operations. We'll now look at a second approach, where we reuse an existing LinkedList class to implement the stack, allowing the stack operations to be built on top of the linked-list functionality we already have.

Implementing a Stack Using a LinkedList Class

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
class LinkedList:
    def __init__(self):
        self.size = 0
        self.head = None
```

```
class Stack:
    def __init__(self):
        self.ll = LinkedList()
```

- The **Node** class represents each node in the linked list.
- The **LinkedList** class manages the nodes using head and size.
- The **Stack** class creates a **LinkedList object** using **self.ll = LinkedList().**
- The stack operations will use this linked list to **store and manage stack elements**.
- The **head of the linked list** can be used to represent the **top of the stack**.

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

This time, we're implementing the stack by reusing a LinkedList class, rather than managing the nodes directly inside the Stack class. The Node class still represents each individual node, while the LinkedList class keeps track of those nodes through its head and size.

When we create a Stack object, the statement self.ll = LinkedList() creates a new linked-list object specifically for that stack. So, self.ll gives the Stack object access to the linked list and the operations that it provides.

From the stack's point of view, the head of this linked list represents the top of the stack. We can therefore implement stack operations such as push() and pop() by using the corresponding linked-list operations, rather than writing the node manipulation again.

Core stack operations



`peek()`: Returns the top element without removing it

`push()`: Adds an element to the top of the stack

`pop()`: Removes and returns the top element

`is_empty()`: Checks if the stack is empty

`get_size()`: Returns the current size of the stack



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Here we have the five core stack operations that we'll implement using our LinkedList object. Although the stack is built on top of a linked list, we still interact with it through familiar stack operations such as push, pop, and peek.

The push() and pop() operations modify the stack, while peek() simply examines the top element without removing it. Since the head of our linked list represents the top of the stack, these operations will work mainly with the beginning of the linked list.

We also need is_empty() to determine whether the stack contains any elements, and get_size() to report how many elements are currently stored. In the next few slides, we'll look at how each of these operations can reuse methods provided by the LinkedList class.

Core stack operations: peek()

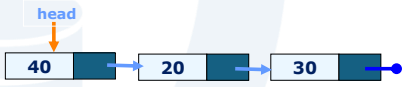
Previous version

```
def peek(self):
    if self.is_empty():
        raise IndexError("Peek from empty stack")
    return self.top.data
```



LinkedList Version

```
def peek(self):
    if self.is_empty():
        raise IndexError("Peek from empty stack")
    return self.ll.head.data
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Here we can see how the peek() operation changes when the Stack uses a LinkedList object internally. In the previous implementation, the stack maintained its own top reference, so we could access the top element directly using self.top.data.

In the new implementation, the Stack no longer has its own top reference. Instead, self.ll refers to the LinkedList object, and the head of that linked list represents the top of our stack, so we access the top element using self.ll.head.data.

The behaviour of peek() itself has not changed: it still returns the top element without removing it. For the example shown, the head contains 40, so peek() returns 40, while the linked list remains unchanged.

Core stack operations: push()

Previous version

```
def push(self, data):
    new_node = Node(data)
    new_node.next = self.top
    self.top = new_node
    self.size += 1
```



New version

```
def push(self, data):
        self.ll.insert_node(0, data)
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The push() operation is considerably simpler in this implementation because the LinkedList class already provides the functionality needed to insert a node. In the previous version, the Stack itself had to create the node, connect it to the current top, update top, and increase the size.

Since the head of the linked list represents the top of the stack, we can perform all of this through self.ll.insert_node(0, data). Index 0 refers to the beginning of the linked list, so the inserted element becomes the new head.

This allows the Stack class to reuse the existing LinkedList operation rather than managing nodes and links itself. From the user's perspective, push() behaves exactly as before: the newly inserted element becomes the top of the stack.

Core stack operations: pop()

Previous version

```
def pop(self):
    if self.is_empty():
        raise IndexError("Pop from empty stack")
    popped_node = self.top
    self.top = self.top.next
    self.size -= 1
    return popped_node.data
```



New version

```
def pop(self):
    if self.is_empty():
        raise IndexError("Pop from empty stack")
    data = self.ll.head.data
    self.ll.remove_node(0)
    return data
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

For pop(), the LinkedList version again allows us to reuse operations that are already available in the LinkedList class. In the previous implementation, the Stack itself had to save the top node, move top to the next node, decrease the size, and return the removed value.

In the new version, the head of the linked list represents the top of the stack. Before removing it, we save its value using data = self.ll.head.data, because we need to return that value after the node has been removed.

We then call self.ll.remove_node(0) to remove the node at index 0, which is the head of the linked list. For the example shown, the node containing 40 is removed, 20 becomes the new head and therefore the new top, and pop() returns 40.

Core stack operations: `is_empty()`

Previous version

```
def is_empty(self):
    return self.top is None
```



New version

```
def is_empty(self):
    return self.ll.size == 0
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

With is_empty(), we again change how the Stack checks its state because the stack is now using a LinkedList object. In the previous version, the Stack maintained its own top reference, so checking self.top is None was enough to determine whether the stack was empty.

In the new version, the LinkedList already maintains a size attribute that records the number of nodes it contains. Therefore, self.ll.size == 0 checks whether the linked list used by this Stack instance contains zero nodes.

This expression directly returns a Boolean value: True when the size is zero and False otherwise. In the example shown, the linked list contains three nodes, so is_empty() returns False.

Core stack operations: get_size()

Previous version

```
def get_size(self):
    return self.size
```



New version

```
def get_size(self):
    return self.ll.size
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

For get_size(), the change is very straightforward because the LinkedList object already keeps track of how many nodes it contains. In the previous version, the Stack maintained its own size attribute, so get_size() simply returned self.size.

In the new version, that responsibility belongs to the LinkedList stored in self.ll. Therefore, we use return self.ll.size, where self.ll refers to the LinkedList object and size gives us the number of nodes currently stored in that linked list.

So we don’t need a separate size counter inside the Stack class. In the example shown, the linked list contains the nodes 40, 20, and 30, so self.ll.size is 3, and get_size() returns 3.

Working Example Using a LinkedList Class

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

We've now seen how the Stack class can reuse a LinkedList object to implement each of the core stack operations. In this working example, we'll bring those pieces together and follow how the complete implementation works, from creating a Stack instance and adding elements to accessing and removing them using the methods we've just discussed.

Working Example – 02

```
class ListNode:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.size = 0
        self.head = None
        self.tail = None

    def find_node(self, index):
        if index < 0 or index >= self.size:
            return None
        temp = self.head
        for _ in range(index):
            temp = temp.next
        return temp

    def remove_node(self, index):
        if index < 0 or index >= self.size:
            return -1
        if index == 0:
            self.head = self.head.next
            if self.size == 1:
                self.tail = None
        else:
            prev = self.find_node(index - 1)
            prev.next = prev.next.next
            if index == self.size - 1:
                self.tail = prev
        self.size -= 1
        return 0
```

```
    def insert_node(self, index, data):
        if index < 0 or index > self.size:
            return -1
        new_node = ListNode(data)
        if index == 0:
            new_node.next = self.head
            self.head = new_node
            if self.size == 0:
                self.tail = new_node
        elif index == self.size:
            self.tail.next = new_node
            self.tail = new_node
        else:
            prev = self.find_node(index - 1)
            new_node.next = prev.next
            prev.next = new_node
        self.size += 1
        return 0
```

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

In this working example, we'll use the LinkedList class that we have already implemented as the underlying structure for our Stack. This slide shows the ListNode and LinkedList classes, including the methods for finding, inserting, and removing nodes.

We don't need to go through these methods line by line again, because we've already covered how the linked list works. What matters here is that the LinkedList class already manages the head, tail, size, and links between nodes.

We can therefore reuse this existing functionality when we build our Stack class. In particular, the stack operations will use insert_node() and remove_node() at the head of the linked list, because the head will represent the top of our stack.

Working Example – 02 (cont.)

```
52  class Stack:
53      def __init__(self):
54          self.ll = LinkedList()
55
56      def push(self, data):
57          self.ll.insert_node(0, data)
58
59      def pop(self):
60          if self.is_empty():
61                 raise IndexError("Pop from empty stack")
62          data = self.ll.head.data
63          self.ll.remove_node(0)
64          return data
65
66      def peek(self):
67          if self.is_empty():
68              raise IndexError("Peek from empty stack")
69          return self.ll.head.data
70
71      def is_empty(self):
72          return self.ll.size == 0
73
74      def get_size(self):
75              return self.ll.size
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

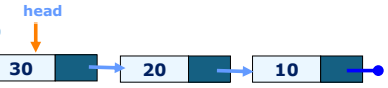
Here we put everything together by defining the Stack class using the LinkedList object. When a Stack instance is created, self.ll = LinkedList() creates the linked list that will actually store and manage the stack elements.

The stack operations then reuse the linked-list functionality we discussed earlier. push() inserts at index 0, while pop() retrieves the item at the head and removes that node; peek() simply returns the item at the head without removing it.

The remaining methods are also straightforward. is_empty() checks whether the linked-list size is zero, and get_size() returns that size, so the Stack class provides stack behaviour while the LinkedList class handles the underlying nodes and links.

Working Example – 02 (cont.)

```
76 if __name__ == "__main__":
77
78     s = Stack()  # Create a new stack
79
80     s.push(10)   # Push some elements: 10, 20, and 30
81     s.push(20)
82     s.push(30)
83
84     print("Size:", s.get_size())      # Print the size. Should print 3
85
86     print("Top element:", s.peek()) # Print top element. Should print 30
87
88     print("Popped:", s.pop())       # Should print 30
89     print("Popped:", s.pop())       # Should print 20
90
91     print("New size:", s.get_size())  # Print new size. Should print 1
92
93     print("New top element:", s.peek()) # Print new top element. Should print 10
94
95     print("Is stack empty?", s.is_empty()) # Check if empty. Should print False
96
97     print("Popped:", s.pop())       # Pop last element. Should print 10
98
99     print("Is stack empty now?", s.is_empty()) # Check if empty again. Should print True
```



© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

In this example, you may notice that the way we use the Stack has not changed at all. We still create s = Stack() and call the same operations such as push(), pop(), peek(), is_empty(), and get_size().

What has changed is only the internal implementation of the Stack. In the first approach, the Stack manages linked nodes directly, whereas in this approach, it uses a LinkedList object to manage those nodes for us.

So from the programmer's point of view, both implementations provide the same Stack interface and the same behaviour. This demonstrates abstraction, because the underlying implementation details are hidden, allowing us to change how the Stack is implemented without changing how it is used.

Stack Applications

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Next, we'll look at some common applications of stacks and see where this data structure appears in practical computing problems. As we go through these examples, think about why the Last In, First Out behaviour makes a stack particularly suitable for each situation.

Stacks in Computer Science

- **Operating Systems:**
  - Function call stack to keep track of active function calls
  - Undo/redo functionality in text editors
  - Memory stack for local variable storage and function execution
- **Programming:**
  - Backtracking algorithms (e.g., solving mazes, recursion, depth-first search)
  - Evaluating expressions (e.g., parsing mathematical expressions using postfix notation)
  - Managing history (e.g., web browser back/forward navigation)
- **Real-World Examples:**
  - A stack of plates in a cafeteria (Last In, First Out - LIFO)
  - Piling books on a table (you take the topmost book first)

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Stacks appear in many areas of computer science because Last In, First Out behaviour naturally fits situations where the most recent item needs to be handled first. For instance, function calls use a stack so that the most recently called function can finish before execution returns to the previous one.

In programming, stacks are useful for backtracking, depth-first search, expression evaluation, and maintaining history. Think about browser navigation: when you move backwards through previously visited pages, the most recent page is the first one you return to.

We can also relate this behaviour to familiar physical objects. A stack of plates or books follows the same principle—you normally remove the item from the top before accessing the ones underneath.

Some use cases of stacks

- **Tracking nested structures**
  - Stacks help match nested structures such as **parentheses and HTML tags**.
- **Evaluating expressions**
  - Stacks support **expression evaluation** and help manage operator precedence and execution order.
- **Managing function or method calls**
  - The **call stack** keeps track of active function calls and where execution should return.
- **Tracking choices or steps**
  - Algorithms such as **depth-first search (DFS)** use stacks to manage choices or steps still to be explored.
- **Supporting backtracking**
  - Stacks store previous states, allowing algorithms to **return to earlier choices** when necessary.

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Stacks appear in many computing problems because they naturally keep track of what was most recently encountered or performed. For example, they can match nested structures such as parentheses and HTML tags, and they are also widely used when evaluating expressions where operators and operands must be processed in the correct order.

Another important application is function execution. A call stack keeps track of active function or method calls, allowing the program to return to the correct point when each call finishes.

Stacks are also useful when algorithms need to explore different possibilities. Depth-first search can store choices that still need to be explored, while backtracking can preserve previous states so the algorithm can return to an earlier choice after reaching a dead end.

Conclusion

© 2026 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

To conclude, we have explored the stack data structure, its LIFO behaviour, core operations, different implementations, and some common applications, and the next slide will summarise the main points from this lecture.

Conclusion

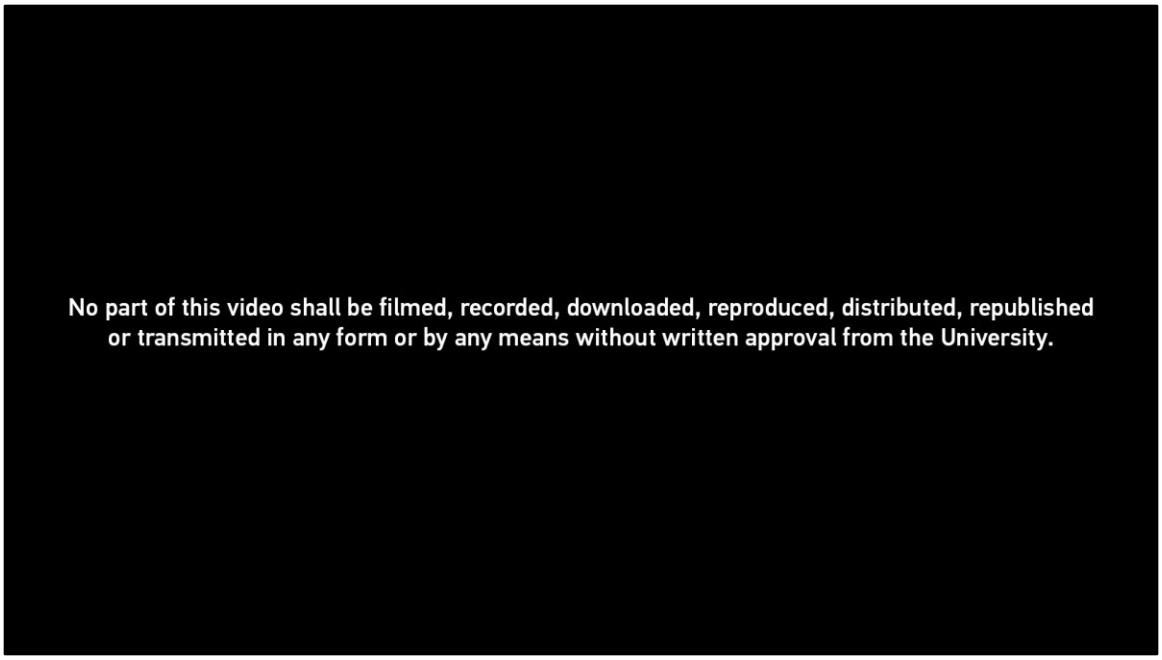
- A Stack is a linear data structure that follows the Last In, First Out (LIFO) principle.
- Elements are added and removed from one end of the stack, called the top.
- Core operations include push, pop, peek, is_empty, and get_size.
- A stack can be implemented directly using linked nodes or by using an existing LinkedList class.
- Stack operations can be implemented efficiently by treating the head of the linked list as the top.
- Stacks support many computing tasks, including function calls, expression processing, recursion, backtracking, and history management.

© 2026 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

To bring everything together, a stack is a linear data structure based on the Last In, First Out principle, where all insertion and removal take place at the top. The operations we've used throughout the lecture—push, pop, peek, is_empty, and get_size—give us a simple interface for working with it.

We've also seen that the same stack behaviour can be implemented in different ways. We can manage the linked nodes directly, or reuse an existing LinkedList class and treat its head as the top of the stack.

Regardless of the implementation, the behaviour presented to the user of the Stack class remains the same. This makes stacks useful in areas such as function calls, recursion, expression processing, backtracking, and history management


No part of this video shall be filmed, recorded, downloaded, reproduced, distributed, republished
or transmitted in any form or by any means without written approval from the University.